

```

print_all_devices(devs);
libusb_free_device_list(devs, 1);
libusb_exit(NULL);
return 0;
}

```

As you can see in this code, we initiated the use of the `libusb_init()` function and called the inbuilt `libusb_get_device_list()` function to get the list. Then we used a custom function, `print_all_devices()`, to print all the devices (`devs`). You may have noticed that this function, in turn, invoked the `libusb_device_descriptor()` to get the descriptors and print them. The only function that is obscure here is `libusb_free_device_list()`, which frees a list of devices previously found. Then, we safely exit the program.

Isn't this quite easy? The only point that you need to be aware of is to add a few declarations and perform some checks for the variables (see the comments in the code).

You might be aware of the fact that all USB devices have their own vendor and product identification values. So, if you want to refine the list to a particular device, all you need to do is perform another check for these two IDs:

```

if ((mydevice->descriptor.idVendor == YOUR_ID1) && (mydevice->descriptor.
idProduct == YOUR_ID2))

{
    return mydevice;
}

```

From our previous discussions, it is clear that this code is returning a pointer to the device (if the device is found). And this actually corresponds to the structure `usb_device` that we saw earlier.

This means that you can now start doing basic operations (you may refer to the file operations we described earlier). Let's say you want to open the device to initiate a 'communication'. You can perform the same, using the following lines of code:

```

usb_opr = usb_open(usb_dev);
if (usb_opr== NULL) {
    fprintf(stderr,
        "Failed to open\n");
    goto exit;
}

```

That's it!

You might find it interesting to note that the `usb_opr` is of the type `struct usb_dev_handle` and thus `libusb` is able to initiate the communication with a particular USB device. The moment you issue that, it will perform all the initial steps required to start the communication.

Once you are done with the communication, you can use the following code to close it safely:

```
usb_close(usb_handle);
```

Error codes returned by `usb_submit_urb`

Non-USB type

- 0 URB submission went fine
- ENOMEM no memory for allocation of internal structures

USB related

- ENODEV specified USB-device or bus doesn't exist
- ENOENT specified interface or endpoint does not exist or is not enabled
- ENXIO host controller driver does not support queuing of this type of urb (treat it as a host controller bug)
- EINVAL
 - a) Invalid transfer type specified (or not supported)
 - b) Invalid or unsupported periodic transfer interval
 - c) ISO: attempted to change transfer interval
 - d) ISO: number_of_packets is < 0
 - e) various other cases
- EAGAIN
 - a) specified ISO started frame too early
 - b) (using ISO-ASAP) too much scheduled for the future, wait some time and try again.
- EFBIG Host controller driver can't schedule that many ISO frames.
- EPIPE Specified endpoint is stalled. For non-control endpoints, reset this status with `usb_clear_halt()`.
- EMSGSIZE
 - (a) endpoint maxpacket size is zero; it is not usable in the current interface's altsetting.
 - (b) ISO packet is larger than the endpoint maxpacket.
 - (c) requested data transfer length is invalid: negative or too large for the host controller.
- ENOSPC This request would overcommit the USB bandwidth reserved for periodic transfers (interrupt, isochronous).
- ESHUTDOWN The device or host controller has been disabled due to some problem that could not be worked around.
- EPERM Submission failed because `urb->reject` was set.
- EHOSTUNREACH URB was rejected because the device is suspended.

It will be helpful if you remember some of the error codes returned by `usb_submit_urb` (in case of an asynchronous transfer request). This will help you to fix bugs in the codes. You can find the details in the box on "Error codes returned by `usb_submit_urb`".

You are now ready to write drivers for your USB devices. Go ahead and give it a try. If you need any further assistance in writing a good USB driver, please let me know.

Happy kernel hacking! 

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com